Discorso di presentazione

Co-evoluzione tra opinioni, rete e fact-checking con agenti LLM · Analisi di Social Network e Media

Durata stimata: circa 11 minuti a un ritmo di 135 parole al minuto. Le indicazioni tra parentesi quadre, in blu, sono azioni sulle slide e non vanno lette.

Slide 1 — Co-evoluzione tra opinioni, rete e fact-checking con agenti LLM

circa 0:40 min

Buongiorno. Presento il progetto che ho svolto per il corso di Analisi di Social Network e Media.

L'idea è simulare una rete sociale in cui le opinioni degli utenti, i legami tra di loro e un intervento di fact-checking cambiano insieme, nello stesso processo. Gli utenti sono rappresentati da agenti basati su un modello linguistico.

I colori dei nodi indicano i quattro stati degli agenti, e li ritroverete in tutta la presentazione: blu i suscettibili, rosso gli infetti, verde i resistenti e arancione i fact-checker.

Slide 2 — Il progetto in breve

circa 0:50 min

Il punto di partenza è un concetto visto nel corso: l'omofilia. Le persone collegate tendono ad avere opinioni simili, e questo può dipendere da due cose: dalla selezione, cioè scegliamo contatti simili a noi, e dall'influenza sociale, cioè ci adeguiamo ai nostri contatti.

Nella realtà le due cose succedono insieme, ma i modelli classici, come Independent Cascade o Linear Threshold, di solito lavorano su una rete che non cambia.

Per questo mi sono posto tre obiettivi: collegare agenti, rete e intervento in un unico ciclo; misurare la segregazione sia dal punto di vista della struttura sia da quello delle opinioni; e stimare l'effetto di un intervento di fact-checking confrontandolo con una simulazione senza intervento.

Slide 3 — Dati e campionamento

circa 0:55 min

Come dati ho usato ogbl-collab, una rete di co-autoraggio dell'Open Graph Benchmark con circa 235 000 autori. Ogni autore ha un vettore di 128 valori che descrive i suoi temi di ricerca, e l'ho usato come rappresentazione iniziale dei nodi.

Simulare un agente per ogni autore non era possibile, quindi ho estratto un sottografo di 5 000 nodi con Forest Fire, che rispetto al campionamento casuale o alla BFS conserva meglio la struttura della rete: il clustering, per esempio, resta quasi uguale. Quello che si perde sono gli hub più grandi.

Le comunità le ho trovate con Label Propagation, che è veloce e non ha parametri: ne vengono fuori 415, e la partizione resta fissa per tutta la simulazione, così le misure di segregazione sono confrontabili nel tempo.

Slide 4 — Un ciclo chiuso a tre livelli

circa 0:50 min

L'architettura ha tre livelli. Il livello cognitivo contiene gli agenti e una macchina a stati che decide quando cambiano opinione. Il livello strutturale contiene una rete neurale su grafi, GraphSAGE, che stima quali legami sono probabili, e il rewiring che modifica la rete. Il livello di intervento sceglie i fact-checker.

[Seguire le frecce] Le transizioni degli agenti modificano le loro rappresentazioni, queste guidano il rewiring, e la nuova rete decide che cosa leggerà ogni agente allo step successivo. Quando gli infetti superano il 40%, parte l'intervento.

La cosa importante è che tutto avviene nello stesso step, e non in fasi separate: se simulassi prima la diffusione e poi cambiassi la rete, perderei proprio l'interazione che volevo studiare.

Slide 5 — Cosa succede in uno step

circa 0:55 min

In pratica ogni step funziona così. Si attiva a caso il 15% dei nodi, cioè 750 agenti. Ognuno riceve un prompt con il suo profilo, il suo stato e gli ultimi messaggi dei vicini, e il modello risponde con un'opinione, un livello di suscettibilità e lo stato che propone. La macchina a stati decide se la transizione avviene.

Poi l'embedding del nodo viene spostato in base al nuovo stato, GraphSAGE viene aggiornato e, ogni due step, il rewiring cambia fino a 50 archi.

Come modello ho usato Llama 3 da 8 miliardi di parametri su un server vLLM, che gestisce le richieste in parallelo. Tutti gli agenti leggono la stessa fotografia della rete, così il risultato non dipende dall'ordine in cui vengono elaborati.

Slide 6 — Diffusione: un modello a soglia

circa 1:00 min

Per la diffusione ho scelto un modello a soglia, il Linear Threshold visto nel corso: un nodo si infetta quando la frazione di vicini infetti supera una soglia.

L'ho preferito a Independent Cascade perché lì basta un solo contatto per essere contagiati, mentre per cambiare opinione di solito servono più conferme. I modelli SIR, invece, non usano la rete, che qui è proprio quello che voglio studiare.

Ho fatto due modifiche al modello classico. La soglia di ogni nodo viene abbassata o alzata in base alla suscettibilità che l'agente dichiara; e il contagio avviene solo se anche l'agente propone di cambiare stato. Così la rete resta necessaria, e il modello linguistico non può infettare un nodo da solo. Le soglie dei fact-checker, il 10% e il 25%, saranno importanti più avanti.

Slide 7 — Dalle opinioni alla rete

circa 1:00 min

Per far influire le opinioni sulla rete ho usato gli embedding dei nodi: quando un agente cambia stato, il suo embedding si sposta, e i nodi con la stessa opinione diventano più simili.

Su questi embedding lavora GraphSAGE, che stima la probabilità di un legame tra due nodi. L'ho scelto perché usa le feature dei nodi, a differenza di metodi come node2vec, ed è induttivo, quindi non va riaddestrato da zero ogni volta che la rete cambia.

Il rewiring aggiunge gli archi con il punteggio più alto tra nodi che hanno un vicino in comune, e per ogni arco aggiunto rimuove quello con il punteggio più basso, così il numero di archi resta costante. Nella prima versione la rete si densificava, e non si capiva più se le metriche cambiavano per le opinioni o solo perché c'erano più archi.

Slide 8 — Fact-checker: perché non CELF

circa 0:55 min

Per scegliere i fact-checker il progetto prevedeva all'inizio l'algoritmo CELF, visto nel corso per l'Influence Maximization.

Il problema è che qui l'effetto di un fact-checker dipende dalle soglie. Per esempio, per recuperare questo nodo infetto con otto vicini ne servono due. Il primo da solo non serve a niente, il secondo sì: il guadagno cresce invece di diminuire, quindi la funzione non è submodulare. CELF si basa proprio su questa proprietà, e qui potrebbe scartare il candidato migliore.

Ho quindi usato un greedy semplice, che a ogni passo ricalcola tutti i guadagni; è comunque veloce, perché ogni nodo dipende solo dai suoi vicini. Per controllare la scelta ho confrontato la stima con il risultato finale: 162 nodi previsti, 160 ottenuti.

Slide 9 — Come è stato condotto l'esperimento

circa 0:50 min

L'esperimento è diviso in fasi. Dopo la preparazione dei dati, la simulazione parte con 750 nodi centrali già infetti, e la co-evoluzione dura 100 step, eseguiti su Kaggle in due sessioni.

Allo step 99 ho salvato un checkpoint, e da lì ho fatto partire due simulazioni di 30 step: una con 20 fact-checker e una senza. Così posso confrontarle allo stesso step, e la differenza è dovuta all'intervento e non all'evoluzione naturale della rete. Un semplice confronto prima e dopo avrebbe mescolato le due cose.

Il seed ricavato dal prompt fa sì che le due simulazioni ricevano le stesse risposte finché i prompt sono uguali. Uno step dura circa 6 minuti, quasi tutti spesi nelle chiamate al modello.

Slide 10 — La diffusione si ferma al 40%

circa 0:45 min

Il primo risultato riguarda la diffusione. *[Indicare il grafico]* Gli infetti, in rosso, crescono velocemente e poi si stabilizzano; i suscettibili, in blu, scendono; i resistenti, in verde, crescono quasi quanto gli infetti.

La curva tratteggiata è una logistica, che descrive molto bene i dati, e si ferma a circa 2 000 infetti, il 40% della rete. A differenza del modello SI, dove alla fine si infettano tutti, qui una parte dei nodi diventa resistente e altri non raggiungono mai la soglia: l'andamento assomiglia quindi più a un modello SIR. Quasi tutto succede nei primi 20 step, poi la rete si stabilizza.

Slide 11 — Echo chamber soprattutto di opinione

circa 0:50 min

Il secondo risultato riguarda la segregazione. *[Indicare il grafico]* Le metriche strutturali, in viola, crescono di poco, intorno al 5–6%. Le metriche di opinione, in arancione, crescono molto di più: l'assortatività delle opinioni aumenta del 145%.

Con il test di omofilia del corso ho confrontato i legami tra opinioni diverse con quelli che ci si aspetterebbe per caso: all'inizio non c'è omofilia, alla fine i legami tra opinioni diverse sono meno di un terzo di quelli attesi. Quindi le echo chamber si formano durante la simulazione, e guardando solo le metriche strutturali le avrei sottostimate.

Un limite di questa analisi è che non riesco a separare quanto dipende dal rewiring e quanto dal contagio.

Slide 12 — Il fact-checking previene, non cura

circa 0:50 min

Il terzo risultato è l'effetto dell'intervento. Rispetto al controllo ci sono 160 resistenti in più, quasi esattamente quanti ne aveva stimati la selezione. Sono soprattutto suscettibili protetti, circa il 16% di quelli rimasti.

Sugli infetti invece l'effetto è praticamente nullo: 4 in meno, che è meno della variazione naturale. Il motivo è nelle regole: per recuperare un infetto serve che un quarto dei suoi vicini sia fact-checker, e con 20 nodi non succede quasi mai.

[Indicare il grafico] Il grafico mostra che l'effetto arriva quasi tutto nei primi 10 step, come previsto dalla probabilità con cui i nodi vengono attivati. In pratica il fact-checking, in questa forma, funziona come un vaccino e non come una cura.

Slide 13 — Limiti e conclusioni

circa 0:55 min

Per concludere, riassumo i risultati: la diffusione si ferma al 40% con un andamento di tipo SIR; la co-evoluzione crea echo chamber soprattutto di opinione; e un intervento tardivo con pochi fact-checker protegge i suscettibili, ma non recupera gli infetti.

Il lavoro ha anche dei limiti. Ogni configurazione l'ho eseguita una sola volta, perché ogni simulazione richiede ore, quindi gli effetti più piccoli andrebbero confermati. Inoltre una rete di co-autoraggio è solo un'approssimazione di una rete sociale, e i risultati dipendono dal modello e dai prompt usati.

Come sviluppi futuri vorrei ripetere le simulazioni con seed diversi, provare varianti del rewiring per separare selezione e influenza, e anticipare l'intervento per vedere se così riesce a ridurre anche gli infetti. Grazie per l'attenzione.